

Test technique

Stage IA

OBJECTIF DE LA MISSION

Construire un script Python qui lit un texte réglementaire, extrait les exigences, les compare à une fiche produit, et produit un rapport indiquant ce qui est conforme, ce qui manque, et ce qui est ambigu, en distinguant les trois.

Duree	2h de travail effectif
Stack	Python 3.10+ . PyTorch ou toute librairie NLP de votre choix à vous de choisir
Remise	Lien GitHub depot public ou acces partage

Aucun fichier à télécharger.

Tout est dans cet énoncé. Lisez les deux textes en entier avant de coder.

Repérez les différences manuellement avant d'écrire la moindre ligne de code.

Le contexte

Chez Optima AI, on automatise la vérification de conformité réglementaire pour les fabricants industriels. Notre outil lit un texte normatif, en extrait les exigences, puis vérifie si la fiche d'un produit les satisfait.

Les données fournies

Lisez les deux textes en entier. Repérez les différences à l'œil avant de coder.

Le texte réglementaire

```
=== EXTRAIT REGLEMENTAIRE SIMPLIFIE (fictif) ===
```

```
REQ-01 : Le fabricant doit rédiger une déclaration CE avant mise sur le  
marché.
```

```
REQ-02 : La déclaration CE doit être signée par un représentant habilité.
```

```
REQ-03 : La machine doit porter le marquage CE avant commercialisation.
REQ-04 : Le dossier technique doit inclure les schemas des circuits de
commande.
REQ-05 : Une notice d'instructions doit accompagner la machine.
REQ-06 : La notice doit etre disponible dans la langue de chaque pays de
commercialisation.
REQ-07 : La notice doit mentionner explicitement les risques residuels
identifies.
REQ-08 : Une evaluation des risques couvrant le cycle de vie complet doit
etre documentee.
REQ-09 : Les elements mobiles dangereux doivent etre proteges par des
dispositifs adequats.
REQ-10 : Un dispositif d'arret d'urgence conforme a EN 13850 doit etre
present.
REQ-11 : Pour les machines a risque eleve, un organisme notifie doit
intervenir.
=== FIN ===
```

La fiche produit a auditer

Attention : certains champs utilisent un vocabulaire different. D'autres sont presents mais insuffisants. Lisez chaque ligne soigneusement.

```
=== FICHE PRODUIT : ROBOT DE SOUDAGE RS-440 ===
Fabricant      : AutoWeld Technologies SAS
Signataire     : Mme Isabelle RENARD, Responsable Qualite
Marches vises  : France, Italie, Portugal

--- DOCUMENTATION ---
Dossier technique : COMPLET
Schemas electriques et pneumatiques : PRESENTS
Note : machine tout-electrique, pas de circuits hydrauliques
Declaration CE : EN COURS, signature prevue avant livraison

--- SECURITE ---
Enceinte de protection avec acces verrouille : OUI
Bouton STOP d'urgence en facade : OUI, certifie EN ISO 13850
Evaluation des risques : REALISEE pour la phase d'utilisation uniquement
Categorie de risque : standard (pas de categorie speciale)

--- MARQUAGE ET CONFORMITE ---
Marquage CE : OUI, appose sur le tableau de commande

--- DOCUMENTATION UTILISATEUR ---
Notice d'instructions : PRESENTE en francais
Contenu notice - mise en service : OUI
Contenu notice - maintenance : OUI
Contenu notice - risques residuels : OUI, section 8.3 (Entretien et
nettoyage)
=== FIN ===
```

Ce qu'on vous demande

Trois etapes. Tout peut tenir dans un seul fichier audit.py.

Etape 1 : Structurer les exigences

Parsez le texte reglementaire. Extrayez chaque exigence (REQ-01 a REQ-11) sous forme structuree en Python. Format libre. Expliquez votre choix dans CHOIX.md.

Etape 2 : Analyser la fiche produit

Parsez la fiche produit et extrayez les informations disponibles de facon structuree. Certains champs utilisent un vocabulaire different des exigences. Votre structure doit permettre la comparaison.

Etape 3 : Comparer et produire le rapport

Pour chaque exigence, determinez : SATISFAIT / NON SATISFAIT / AMBIGU. La categorie AMBIGU est obligatoire dans votre rapport — elle signale les cas ou l'information est presente mais insuffisante pour conclure.

```
$ python audit.py

> RAPPORT D'AUDIT - RS-440
> Satisfait      : X / 11
> Non satisfait  : X / 11
> Ambigu        : X / 11

> NON SATISFAIT :
> [REQ-XX] Description + raison

> AMBIGU (information presente mais insuffisante) :
> [REQ-XX] Description + ce qui manque pour conclure

> SATISFAIT :
> [REQ-XX] ...
```

Sur la methode de comparaison : approche libre

Matching mots-cles, similarite semantique PyTorch, LLM... vous choisissez.

Ce qu'on regarde en priorite : est-ce que votre outil sait dire AMBIGU quand c'est ambigu ? C'est ca la vraie difficulte de cet exercice.

Livrables a remettre

Fichier	Contenu attendu
<code>audit.py</code>	Pipeline complet : lecture, structuration, comparaison, rapport
<code>requirements.txt</code>	Dependances uniquement
<code>CHOIX.md</code>	Methode, gestion des ambiguïtes, ce qui vous a bloqué, ameliorations
<code>README.md</code>	Une commande pour lancer l'audit

Si vous manquez de temps

Priorite : etapes 1 et 2 propres + la categorie AMBIGU dans le rapport.
Deux categories bien gerees valent mieux que onze categories approximatives.
Indiquez dans CHOIX.md ce que vous n'avez pas eu le temps de faire.

Ce qu'on evalue

Critere	Ce qu'on observe	Poids
Precision de la lecture	Les cas ambigus sont identifiés, pas seulement les cas evidents	30 %
Qualite du code	Lisible, structure, chaque fonction fait une chose precise	25 %
Pipeline qui tourne	<code>audit.py</code> produit un rapport complet en une execution	25 %
CHOIX.md	Methode expliquee, blocages documentes, limites honnetes	20 %